

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

PostgreSQL Views, PL/pgSQL, Functions, Procedures, and Triggers

Reusable Database-Side Logic, Integrity Enforcement, and Database-vs-API Boundaries

Course	IT 108 - Advanced Database Management System
Activity	Laboratory Activity No. 5
Output	Project views, PL/pgSQL functions, stored procedure(s), triggers, test evidence, and database-vs-API logic justification
Maximum Score	80 points

Student / Project Information

Block: C1	Block: C1 Date: September 28, 2026
System Title: BoardMap: A GIS-Based Machine Learning Boarding House Mapping and Recommendation System in Cabadbaran City	Instructor: Jean Y. Villasin, MIT
MEMBERS: Kim L. Embarnace Leojnn August Mig Sabayle Alexander Camacho	

CARAGA STATE UNIVERSITY - CABADBARAN CAMPUS IT 108 | LAB ACTIVITY 5 **1. Activity Alignment and Purpose**

Topic Learning Outcome: Implement reusable PostgreSQL-side logic where appropriate to enforce data integrity and support the semester system without duplicating application-layer business logic.

Course Outcome Alignment: Implement database-side logic using PostgreSQL views, PL/pgSQL functions, stored procedures, and triggers, and justify which business rules should remain in the REST API/backend layer.

Syllabus Topics: Views and materialized views; PL/pgSQL structure; user-defined functions; stored procedures; triggers and trigger functions; exception handling; and choosing between database-side logic and REST API/backend business logic.

Syllabus Assessment: Project database views/functions/procedures/triggers and database-vs-API logic justification.

Purpose of the Activity

Your team will add reusable database-side logic to the PostgreSQL schema of your semester project. The goal is not to move all business logic into the database. Instead, you will identify rules that are best enforced close to the data, implement them with the appropriate PostgreSQL object, test their behavior, and document why other rules should remain in the REST API/backend layer. The resulting database logic must improve integrity, reuse, reporting, or transaction safety without creating hidden or duplicated behavior.

Important distinction: A view, function, procedure, or trigger is justified only when it has a clear responsibility. Avoid duplicating the same rule in PostgreSQL and the backend unless the duplicate check is intentionally defensive and documented.

2. Intended Learning Objectives

- Create regular views that simplify secure, reusable, or report-oriented access to project data.
- Evaluate when a materialized view is appropriate and explain its refresh and staleness implications.
- Write PL/pgSQL functions with parameters, local variables, control structures, return values, and reusable database logic.
- Create stored procedures for appropriate multi-step database operations and explain how procedure calls differ from ordinary SELECT queries.
- Implement trigger functions and triggers for integrity, audit, lifecycle, or derived-data rules that should execute automatically.
- Use exception handling and meaningful error messages for predictable database-side failures.
- Decide whether each important business rule belongs in PostgreSQL, the REST API/backend, or both as a deliberate safeguard.
- Test and document database-side logic using reproducible SQL evidence connected to actual semester-system features.

3. Activity Specifications

Item	Specification
Mode	Existing semester-project group; all members must contribute SQL and must be able to explain the implemented database logic.
Suggested Time	One laboratory session of approximately 3 hours for guided implementation, plus project refinement and presentation.
Primary Output	A project database-logic portfolio containing views, PL/pgSQL function(s), stored procedure(s), trigger(s), and testing evidence.
Supporting Output	Database-vs-API logic justification matrix, object-to-feature traceability, and group presentation/oral defense.
Required Platform	PostgreSQL using pgAdmin, psql, DBeaver, Supabase SQL Editor, or another approved PostgreSQL client.
Presentation Format	5-7 minutes per group plus 2-3 minutes of questions. Every member must explain at least one database object or boundary decision.

- Final or current PostgreSQL schema from the semester system, including primary keys, foreign keys, and important integrity constraints.
- Approved list of users/roles, major features, and important workflows that read or modify database records.
- At least five

candidate business rules that may require reusable queries, integrity enforcement, calculations, audit behavior, or multi-step transactions.

- Access to the project PostgreSQL database and permission to create views, functions, procedures, and triggers in the development environment.
- A backup or SQL export of the current schema/data before testing triggers or routines that modify records.
- Lecture notes on views/materialized views, PL/pgSQL, functions, procedures, triggers, exception handling, and database-vs backend design boundaries.

Safety Rule for Development Data

Perform trigger and procedure testing only on a development database or a safe copy of project data. Do not test destructive logic on a live production database. Use transactions and rollback during early testing when appropriate.

5. Part A - Database Logic Boundary Analysis

Before writing SQL, identify project rules and decide where each rule belongs. Consider data integrity, reuse across clients, transaction atomicity, performance, security, maintainability, and the need for application context or external services.

No.	Feature / Business Rule	Candidate Location	Why This Location?	Risk of Duplication	Evidence / Requirement
1	Room availability/status must remain valid	PostgreSQL	CHECK/status rules plus trigger protect the data regardless of client.	API-only validation could be bypassed by another writer.	boarding_houses + rooms
2	Owner may modify/deactivate only their own listing	REST API/backend	Authenticated user/session and ownership context belong in the backend.	Database-only session logic would couple PostgreSQL to application auth.	Owner listing workflow
3	Approved listing/search projection	PostgreSQL view	Reusable joins and filters can serve Flutter/API clients consistently.	Duplicated SQL can produce inconsistent listing data.	Map & Search
4	Deactivate a house and its rooms together	PostgreSQL procedure	One database command handles the multi-step state change.	Separate API calls can leave partial state.	Owner Manage Listing
5	Room status follows available slots	PostgreSQL trigger	Row-change lifecycle rule should run regardless of writer.	Another writer could bypass API-only logic.	Room lifecycle
6	ML recommendation scoring/personalization	REST API/backend	Model execution and application context belong in backend/ML; DB stores results.	DB routines should not depend on external model runtime.	Recommendation Results
7	Review rating 1–5 and one review per user/house	PostgreSQL constraints	CHECK and UNIQUE are authoritative; API pre-validates for UX.	API-only checks can be bypassed by direct DB writes.	Reviews/Ratings
8	Approved listing core edits require re-review	PostgreSQL trigger	Core-field changes can automatically return an approved listing to pending.	API-only enforcement can be bypassed by another writer.	Listing approval lifecycle

Boundary Decision Guide

Decision	Typical Reasoning
Prefer PostgreSQL	Integrity rules that must hold regardless of which client writes data; reusable data transformations; database centered summaries; atomic data operations; audit/history capture tied directly to table changes.
Prefer REST API / backend	Rules that require external APIs, ML models, file processing, complex authorization context, user-session behavior, orchestration across services, or application-specific presentation logic.
Use both carefully	A backend validation may improve user feedback while a database constraint/trigger provides final protection. If both exist, document which layer is authoritative and keep behavior consistent.
Avoid triggers for	Ordinary application workflow that should be explicit and easy to trace, external network calls, or logic that becomes surprising when a row is modified.

F-CID-010 Rev.3, 03-21-2025 Competent - Service-oriented - Upright Page 3 of 12
CARAGA STATE UNIVERSITY - CABADBARAN CAMPUS IT 108 | LAB ACTIVITY 5

The first screenshot shows a query executed in pgAdmin 4:

```
SELECT *
FROM system_core.vw_approved_boarding_houses;
```

The results are displayed in a table with 5 rows:

boarding_house_id	house_name	description	capacity	contact_number	own
1	Calamba Student Residence	Affordable boarding house suitable for students, located near the university and public transport...	12	09261234576	Rob
2	Bayang Comfort Boarding Hou...	Quiet and clean boarding house offering furnished rooms and basic facilities for students.	10	09271234577	Elen
3	Kauswagan Student Lodge	Student-friendly accommodation with Wi-Fi, parking space, and shared kitchen facilities.	15	09281234578	Ped
4	Poblacion Study House	Convenient boarding house near stores, transportation, and other essential establishments.	10	09301234580	Sof
5	Model Student Residence	Model Student Residence located near the university and public transport...	10	09261234576	Rob

The second screenshot shows a more complex query:

```
SELECT *
FROM system_core.vw_approved_boarding_houses;
```

The results are displayed in a table with 5 rows:

contact_number	owner_name	owner_contact	barangay_name	city	province	latitude	longitude
09261234576	Robert Menalo	09211234571	Calamba	Cabadbaran City	Agusan del Norte	9.1225000	125.5350000
09271234577	Elena Garcia	09221234572	Bayang	Cabadbaran City	Agusan del Norte	9.1168000	125.5350000
09281234578	Pedro Villanueva	09231234573	Kauswagan	Cabadbaran City	Agusan del Norte	9.1197000	125.5421000
09301234580	Sofia Mendoza	09251234575	Poblacion 2	Cabadbaran City	Agusan del Norte	9.1239000	125.5326000
09261234576	Robert Menalo	09211234571	Calamba	Cabadbaran City	Agusan del Norte	9.1225000	125.5350000

6. Part B - Views and Materialized Views

Create views that make repeated project queries easier to use and maintain. A view should have a clear consumer, such as an API endpoint, admin dashboard, report, or role-limited data access pattern.

Required Item	Minimum Evidence
Regular View 1	A reusable project view that joins or filters data needed by a real feature, report, or API endpoint.
Regular View 2	A second view with a different purpose, such as a summary, status list, user-facing listing, or admin/reporting view.
Materialized View Evaluation	Identify one candidate reporting/aggregation query and decide whether to materialize it. Implement only when the performance benefit justifies refresh/staleness complexity.
Security / Exposure Review	Verify that no view exposes sensitive columns that the consuming feature should not receive.
Traceability	Map each view to the requirement, feature, or report that consumes it.

View Design Worksheet

ID	View Name	Feature / Consumer	Base Tables	Purpose / Columns Exposed	Materialized? Refresh Plan
V01	vw_approved_boarding_houses	Student Map/Search API	boarding_houses; boarding_house_owners; locations	Approved listing, owner and map-location fields	No
V02	vw_available_boarding_rooms	Student Search / Recommendation API	boarding_houses; rooms; locations	Approved houses with available room type, price, capacity, slots and map data	No
V03	mv_boarding_house_rating_summary	Admin/reporting candidate	boarding_houses; reviews	Average rating and review count by approved house	Evaluate; implement only if justified

Generic SQL Pattern - Replace with Your Project Names

```
CREATE OR REPLACE VIEW system_core.vw_approved_boarding_houses AS
SELECT bh.boarding_house_id, bh.house_name, bh.description,
       bh.capacity, bh.contact_number,
       bho.name AS owner_name, bho.contact_number AS owner_contact,
       l.barangay_name, l.city, l.province, l.latitude, l.longitude
FROM system_core.boarding_houses bh
JOIN system_core.boarding_house_owners bho ON bho.owner_id=bh.owner_id
JOIN system_core.locations l ON l.location_id=bh.location_id
WHERE bh.status='approved';
SELECT a.id, a.status, b.name AS category_name
FROM project_schema.entity_a a
JOIN project_schema.entity_b b ON b.id = a.category_id
WHERE a.is_active = true;
```

```
-- Use a materialized view only when justified:
-- Candidate only; implement only if admin reporting justifies stale cached results.
```

```
CREATE MATERIALIZED VIEW system_core.mv_boarding_house_rating_summary AS
SELECT bh.boarding_house_id, bh.house_name,
       ROUND(AVG(rv.rating),2) AS average_rating,
       COUNT(rv.review_id) AS review_count
FROM system_core.boarding_houses bh
LEFT JOIN system_core.reviews rv ON rv.boarding_house_id=bh.boarding_house_id
```

```

WHERE bh.status='approved'
GROUP BY bh.boarding_house_id,bh.house_name;
SELECT date_trunc('month', created_at) AS month_key, count(*) AS total
FROM project_schema.entity_a
GROUP BY 1;

```

Materialized View Reminder

Materialized views return stored results and can become stale. If used, document who/what refreshes the view, how often refresh occurs, and whether stale data is acceptable for the feature.

F-CID-010 Rev.3, 03-21-2025 Competent - Service-oriented - Upright Page 4 of 12

CARAGA STATE UNIVERSITY - CABADBARAN CAMPUS IT 108 | LAB ACTIVITY 5

The first screenshot shows the creation of a function `fn_get_available_room_count` in the `system_core` schema. The function takes a boarding house ID as input and returns the count of available rooms. The query is executed successfully, returning 1 row.

```

CREATE OR REPLACE FUNCTION system_core.fn_get_available_room_count(
    p_boarding_house_id BIGINT
)
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN (
        SELECT COALESCE(SUM(r.available_slots), 0)::INTEGER
        FROM system_core.rooms AS r
        WHERE r.boarding_house_id = p_boarding_house_id
        AND r.status = 'available'
    );
END;

```

The second screenshot shows the creation of a materialized view `fn_is_boarding_house_available` in the `public` schema. The view checks if a boarding house is available based on the function. The query is executed successfully, returning 1 row with the value `true`.

```

RAISE EXCEPTION 'Boarding house ID % does not exist.', p_boarding_house_id;
END IF;
RETURN EXISTS (
    SELECT 1
    FROM system_core.rooms AS r
    WHERE r.boarding_house_id = p_boarding_house_id
    AND r.status = 'available'
    AND r.available_slots > 0
);
END;

```

The third screenshot shows the execution of a query that calls the materialized view function. The query is executed successfully, returning 1 row with the value `true`.

```

SELECT system_core.fn_is_boarding_house_available(1);

```

7. Part C - PL/pgSQL User-Defined Functions

Implement reusable functions for logic that belongs close to the data. Functions should have a single clear responsibility and a predictable return type. Avoid placing unrelated workflow logic into one large function.

Requirement	Evidence for Completion
Function 1	Parameterized scalar or validation/calculation function tied to a real project rule.
Function 2	A function that returns a set/table or encapsulates a reusable data retrieval/transformation required by a feature.
PL/pgSQL Structure	Use DECLARE when local variables are needed; use BEGIN/END; use IF/ELSIF/CASE/LOOP only when the rule requires control flow.
Volatility / Effects	Avoid hidden writes inside read-oriented functions. Be able to explain whether the function is read-only or data modifying.
Error Behavior	For invalid inputs, return a defensible result or raise a meaningful exception as appropriate.

Function Planning Worksheet

ID	Function Name	Parameters	Return Type	Business Rule / Feature	Why Database-Side?
F01	fn_get_available_room_count	p_boarding_house_id INT	INTEGER	Total available room slots for a boarding house.	Reusable availability calculation.
F02	fn_is_boarding_house_available	p_boarding_house_id INT	BOOLEAN	TRUE only when house is approved and has available slots.	Reusable availability rule.

Generic PL/pgSQL Function Pattern

```
CREATE OR REPLACE FUNCTION system_core.fn_is_boarding_house_available(p_boarding_house_id INT)
RETURNS BOOLEAN LANGUAGE plpgsql STABLE AS $$
DECLARE v_approved BOOLEAN; v_slots INTEGER;
BEGIN
    SELECT (bh.status='approved') INTO v_approved
    FROM system_core.boarding_houses bh
    WHERE bh.boarding_house_id=p_boarding_house_id;
    IF v_approved IS NULL THEN
        RAISE EXCEPTION 'Boarding house % was not found',p_boarding_house_id
        USING ERRCODE='P0002';
    END IF;
    SELECT system_core.fn_get_available_room_count(p_boarding_house_id) INTO v_slots;
    RETURN v_approved AND v_slots>0;
END; $$;

CREATE OR REPLACE FUNCTION system_core.fn_get_available_room_count(p_boarding_house_id INT)
RETURNS INTEGER LANGUAGE plpgsql STABLE AS $$
DECLARE v_slots INTEGER;
BEGIN
    SELECT COALESCE(SUM(r.available_slots),0) INTO v_slots
    FROM system_core.rooms r
    WHERE r.boarding_house_id=p_boarding_house_id AND r.status='available';
    RETURN v_slots;
END; $$;
RETURNS text
LANGUAGE plpgsql
AS $$
DECLARE
    v_status text;
```

BEGIN

```
SELECT status INTO v_status  
FROM project_schema.entity_a  
WHERE id = p_entity_id;
```

IF v_status IS NULL THEN

```
RAISE EXCEPTION 'Entity % was not found', p_entity_id;  
END IF;  
RETURN v_status;  
END;  
$$;
```

Function Design Rule

Do not use a function merely to hide a simple one-line SELECT. A function should improve reuse, enforce a stable rule, simplify a recurring transformation, or provide a meaningful database abstraction.

F-CID-010 Rev.3, 03-21-2025 Competent - Service-oriented - Upright Page 5 of 12

The first screenshot shows a SQL query in the Query Editor window. The query is a PL/pgSQL function that checks if a boarding house exists and updates its status. The query is as follows:

```
BEGIN  
IF NOT EXISTS (  
    SELECT 1  
    FROM system_core.boarding_houses  
    WHERE boarding_house_id = p_boarding_house_id  
    AND owner_id = p_owner_id  
) THEN  
    RAISE EXCEPTION  
        'Boarding house % was not found or does not belong to owner %',  
        p_boarding_house_id, p_owner_id;  
END IF;  
UPDATE system_core.boarding_houses  
SET status = 'Inactive'  
WHERE boarding_house_id = p_boarding_house_id;  
UPDATE system_core.rooms  
SET status = 'Inactive'  
WHERE boarding_house_id = p_boarding_house_id;  
END  
$$
```

The second screenshot shows the results of the query. The Data Output window displays a single row of data:

boarding_house_id	room_id	room_type	price	available_slots	status
1	1	Single	4500.00	1	available

The third screenshot shows the results of a query that calls a function to deactivate a boarding house. The Data Output window displays a single row of data:

boarding_house_id	house_name	owner_id	status
1	Colombia Student Residences	1	inactive

8. Part D - Stored Procedures

Create at least one stored procedure for a project operation that benefits from an explicit database-side command. Suitable cases may include a controlled multi-step update, maintenance task, batch operation, archival workflow, or administrative operation that should execute close to the data.

ID	Procedure Name	Parameters	Multi-Step Operation	Tables Affected	Why Procedure?
P01	sp_deactivate_boarding_house	p_boarding_house_id INT, p_owner_id INT	Verify owner; deactivate house; deactivate its rooms.	boarding_houses; rooms	Multi-step lifecycle operation callable by REST API.

Procedure Suitability Checklist

- The procedure performs a meaningful operation, not merely a wrapper around a single trivial statement.
- The affected tables and transaction expectations are understood.
- The procedure does not depend on Flutter/UI state or external web services.
- The REST API/backend can call the procedure explicitly and handle success/failure correctly.
- The procedure does not silently duplicate another backend workflow unless the database procedure is the authoritative implementation.

Generic Stored Procedure Pattern

```
CREATE OR REPLACE PROCEDURE project_schema.sp_close_record(  
  p_record_id bigint,  
  p_closed_by bigint  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  UPDATE project_schema.entity_a  
  SET status = 'CLOSED', updated_at = now()  
  WHERE id = p_record_id;  
  
  INSERT INTO project_schema.audit_log(entity_id, action, actor_id)  
  VALUES (p_record_id, 'CLOSE', p_closed_by);  
END;  
$$;  
  
CALL project_schema.sp_close_record(1001, 7);
```

Procedure vs Function

Use a function when a returned value/table is central to the operation and it belongs in SQL expressions. Use a procedure for an explicit command or multi-step operation where CALL semantics are clearer. Base the decision on the project need, not on forcing both objects into the same role.

9. Part E - Triggers and Trigger Functions

Triggers execute automatically in response to table events. Use them for rules that must happen reliably whenever the underlying data changes, regardless of which application path performs the change. Because triggers are implicit, every trigger must be clearly documented.

Required Item	Minimum Evidence
Trigger 1 - Integrity / Lifecycle	Enforce or maintain a rule that depends on a row change and cannot be expressed more clearly with a simple constraint.
Trigger 2 - Audit / History or Derived State	Record a meaningful audit/history event or maintain a justified derived state when data changes.
Trigger Function	Each trigger must call a clearly named trigger function with a single documented purpose.
Timing / Event	Choose BEFORE or AFTER and INSERT/UPDATE/DELETE deliberately. Be able to justify the choice.
Scope	Use FOR EACH ROW or statement-level behavior based on the intended rule; avoid unnecessary row level work on high-volume operations.

Trigger Planning Worksheet

ID	Trigger Name	Table / Event	Timing	Trigger Function	Rule Enforced	Why Not API Only?
T01	trg_room_status_from_slots	rooms / INSERT or UPDATE	BEFORE	tg_room_status_from_slots	0 slots→full; positive slots with full→available.	Must apply to every writer.
T02	trg_reapproval_on_listing_change	boarding_houses / UPDATE core fields	BEFORE	tg_reapproval_on_listing_change	Approved listing core edit→pending for re-review.	Should not depend only on one UI/API path.

Generic Trigger Pattern

```
CREATE OR REPLACE FUNCTION system_core.tg_room_status_from_slots()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.available_slots=0 THEN NEW.status:='full';
  ELSIF NEW.available_slots>0 AND NEW.status='full' THEN NEW.status:='available';
  END IF;
  RETURN NEW;
END; $$;
```

```
CREATE TRIGGER trg_room_status_from_slots
BEFORE INSERT OR UPDATE OF available_slots,status ON system_core.rooms
FOR EACH ROW EXECUTE FUNCTION system_core.tg_room_status_from_slots();
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
  NEW.updated_at := now();
  RETURN NEW;
END;
$$;
```

```
CREATE TRIGGER trg_entity_a_updated_at
BEFORE UPDATE ON project_schema.entity_a
FOR EACH ROW
EXECUTE FUNCTION project_schema.tg_set_updated_at();
```

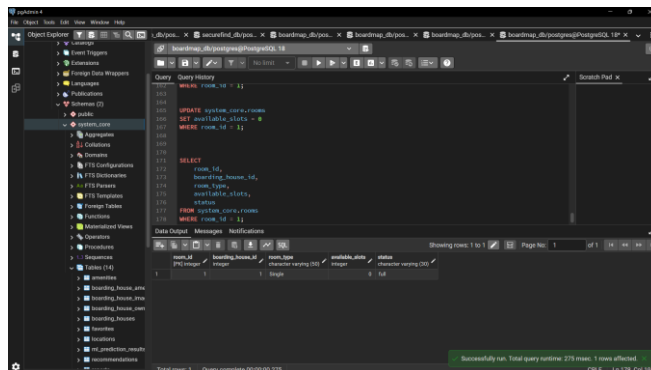
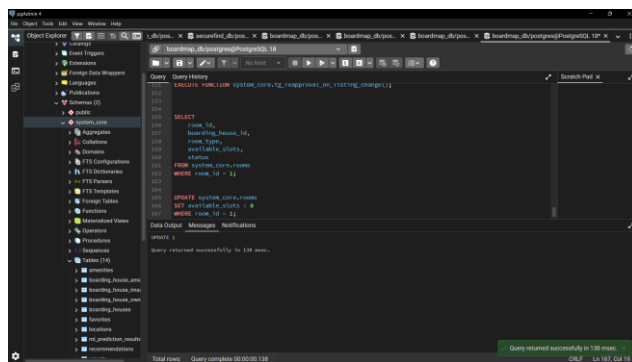
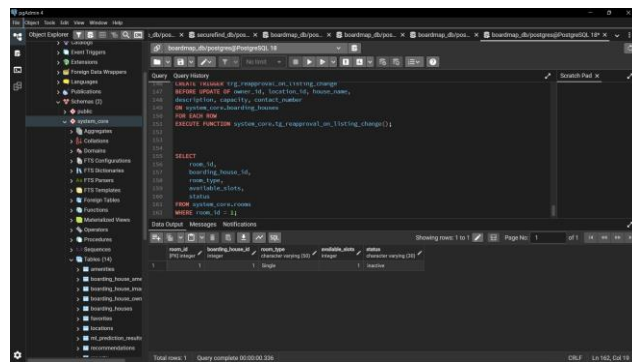
Do Not Overuse Triggers

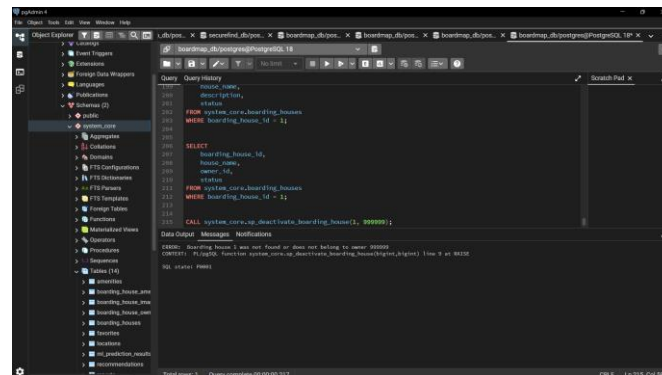
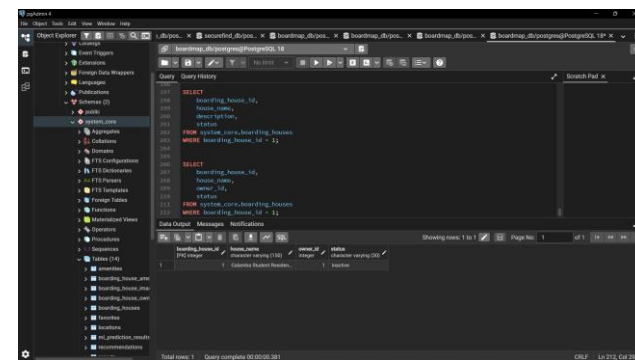
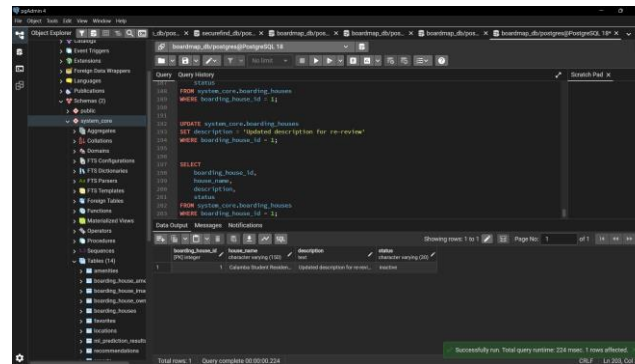
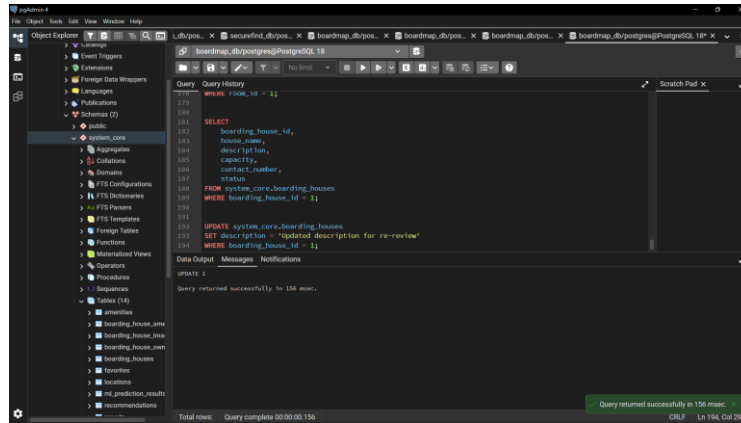
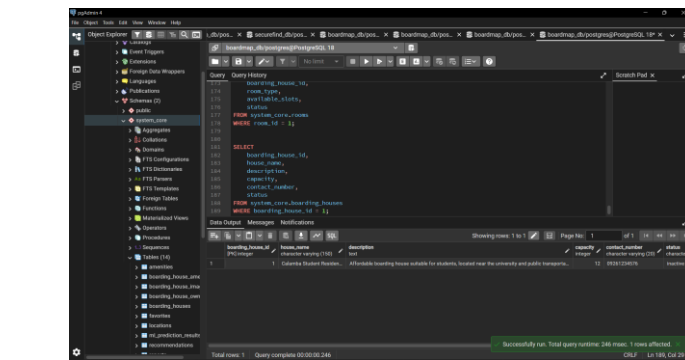
Prefer declarative constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK when they can express the rule directly. Use a trigger only when automatic event-driven behavior is genuinely needed.

F-CID-010 Rev.3, 03-21-2025 Competent - Service-oriented - Upright Page 7 of 12

```
CREATE OR REPLACE FUNCTION system_core.tg_reapproval_on_listing_change()
RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    IF OLD.status='approved' AND (
        NEW.owner_id IS DISTINCT FROM OLD.owner_id OR
        NEW.location_id IS DISTINCT FROM OLD.location_id OR
        NEW.house_name IS DISTINCT FROM OLD.house_name OR
        NEW.description IS DISTINCT FROM OLD.description OR
        NEW.capacity IS DISTINCT FROM OLD.capacity OR
        NEW.contact_number IS DISTINCT FROM OLD.contact_number
    ) THEN
        NEW.status:='pending';
    END IF;
    RETURN NEW;
END; $$;
```

```
CREATE TRIGGER trg_reapproval_on_listing_change
BEFORE UPDATE OF owner_id,location_id,house_name,description,capacity,contact_number
ON system_core.boarding_houses
FOR EACH ROW EXECUTE FUNCTION system_core.tg_reapproval_on_listing_change();
```





10. Part F - Exception Handling and Data Integrity

At least one project routine must demonstrate deliberate error handling. Errors should help the API/backend distinguish invalid input, missing data, state conflicts, or other expected failure conditions.

Failure Type	Expected Handling
Invalid input	Reject values that violate the routine's preconditions; use clear messages and database constraints when possible.
Record not found	Detect missing records when the operation requires an existing target.
State conflict	Prevent an operation that is invalid for the current record status or lifecycle state.
Constraint violation	Allow PostgreSQL constraints to remain authoritative; catch and translate only when the routine truly needs controlled handling.
Unexpected failure	Do not silently swallow errors. Preserve useful failure behavior for debugging and API error handling.

Generic Exception Pattern

```
BEGIN
-- operation here
IF some_condition THEN
RAISE EXCEPTION USING
MESSAGE = 'Operation not allowed for the current state',
ERRCODE = 'P0001';
END IF;
EXCEPTION
WHEN unique_violation THEN
RAISE EXCEPTION 'The business identifier already exists';
END;
```

Error Message Rule

Messages must be understandable to developers and safe for application handling. Do not expose secrets, passwords, tokens, or unnecessary internal details in database errors returned to clients.

11. Part G - Database-vs-API Logic Justification

Complete the matrix for at least five project rules. The purpose is to show that your team understands the boundary between PostgreSQL-side logic and REST API/backend logic.

No.	Rule / Workflow	Authoritative Layer	Supporting Layer	Reasoning	Failure if Duplicated / Misplaced
1	Room availability/status synchronization	PostgreSQL	REST API/backend	Trigger guarantees row-change rule; API validates first.	Layer disagreement if API assumes a different status.
2	Owner-only listing update/deactivation	REST API/backend	PostgreSQL	Auth/session and ownership context belong in backend.	Putting session logic in DB creates coupling.
3	Approved listing/search projection	PostgreSQL view	REST API/backend	Reusable joins/filtering support Flutter and future clients.	Duplicated SQL can produce inconsistent results.
4	Deactivate house + rooms	PostgreSQL procedure	REST API/backend	One command performs related updates together.	Separate calls can leave partial state.
5	ML recommendation generation	REST API/backend	PostgreSQL storage/query	Model execution/application context belongs in backend/ML.	DB routine would depend on external model runtime.

6	Review rating/uniqueness	PostgreSQL constraints	REST API/backend	CHECK/UNIQUE are authoritative; API pre-validates.	API-only checks can be bypassed.
---	--------------------------	------------------------	------------------	--	----------------------------------

12. Project Database-Logic Portfolio Requirements

Portfolio Component	Minimum Project Evidence
Views	At least 2 regular views linked to actual project features; evaluate at least 1 materialized-view candidate and implement it only if justified.
Functions	At least 2 user-defined functions, including at least 1 parameterized PL/pgSQL function. Functions must have distinct project purposes.
Stored procedure	At least 1 procedure for a meaningful multi-step, maintenance, administrative, or transaction oriented operation.
Triggers	At least 2 triggers with clearly documented trigger functions and event/timing justification. At least 1 should protect integrity/lifecycle or audit/history behavior.
Exception handling	At least 1 implemented PL/pgSQL routine demonstrates meaningful handling of an expected failure condition.
Boundary justification	At least 5 rules analyzed for PostgreSQL vs REST API/backend placement.
Testing evidence	At least 10 test cases, including normal, boundary, and failure cases. Include SQL, expected result, actual result, and evidence.
Traceability	Every submitted object maps to a real project feature, requirement, report, integrity rule, or operational need.

Project-Specific Rule

Object counts are minimum evidence, not a license to create meaningless SQL objects. Each object must be technically correct, used or plausibly consumed by the project, and defended during presentation. Redundant or unused objects may not receive full credit.

13. Object-to-Feature Traceability Worksheet

No.	Database Object	Object Type	Project Feature / Requirement	Caller / Consumer	Evidence / File
1	vw_approved_boarding_houses	View	Map/Search approved listings	REST API listing service	SQL + result evidence
2	vw_available_boarding_rooms	View	Search available rooms	REST API search service	SQL + result evidence
3	fn_get_available_room_count	Function	Availability summary	API/reporting	Function SQL + tests
4	fn_is_boarding_house_available	Function	Availability eligibility	API/search	Function SQL + tests

5	sp_deactivate_boarding_house	Procedure	Owner deactivates listing	Owner API service	Procedure SQL + tests
6	tg_room_status_from_slots	Trigger	Room lifecycle status	rooms writes	Trigger SQL + boundary test
7	tg_reapproval_on_listing_change	Trigger	Re-review after listing edit	Owner update workflow	Trigger SQL + state test
8	mv_boarding_house_rating_summary	Materialized View candidate	Admin rating report	Admin reporting	Evaluation; not implemented by default

BoardMap implementation note: create views first, then functions, then the procedure, then trigger functions/triggers. Test on a development copy and keep the REST API as the authority for authentication/authorization and ML orchestration.

14. Laboratory Procedure

Phase 1 - Review system rules and boundaries (20 minutes)

List recurring queries, integrity rules, calculations, audit needs, and multi-step operations. Complete Part A before coding.

Phase 2 - Implement project views (30 minutes)

Create and test at least two regular views. Evaluate a materialized-view candidate and document the refresh decision.

Phase 3 - Live coding / PL/pgSQL functions (40 minutes)

Implement project-specific functions using appropriate parameters, return types, control flow, and error behavior.

Phase 4 - Implement the stored procedure (25 minutes)

Create a meaningful procedure and test the affected rows before and after execution.

Phase 5 - Implement trigger functions and triggers (35 minutes)

Create the required triggers. Test INSERT/UPDATE/DELETE behavior as applicable and verify that the trigger does not create unintended side effects.

Phase 6 - Test failures and exceptions (20 minutes)

Run invalid, missing-record, duplicate, or state-conflict cases. Confirm that errors are predictable and understandable.

Phase 7 - Database-logic boundary discussion and refinement (20 minutes)

Complete the database-vs-API matrix. Remove or redesign logic that is duplicated, hidden, or placed in the wrong layer.

Phase 8 - Export, commit, and submit (10 minutes)

The image displays three sequential screenshots of a PostgreSQL IDE interface, showing the execution of SQL queries and the resulting data output.

Query 1: A query to select boarding house details for a specific house ID.

```
CALL system_core.sp_deactivate_boarding_house(1, '999999');

SELECT
  boarding_house_id,
  house_name,
  owner_id,
  status
FROM system_core.boarding_houses
WHERE boarding_house_id = 1;
```

Data Output:

boarding_house_id	house_name	description	contact_number	owner_name
1	Bayang Comfort Boarding H...	Color and clean boarding house offering furnished rooms and basic facilities for study...	10	Elena Garcia
2	Kauwagan Student Lodge	Student friendly accommodation with Wi-Fi, parking space, and shared kitchen facilities...	10	Pedro Villanueva
3	Pobolacion Study House	Convenient boarding house near stores, transportation, and other essential amenities...	10	Sofia Mendez
4	Matani Budget Rooms	Budget friendly rooms designed for students looking for affordable monthly accommo...	8	Ramon Castillo

Query 2: A query to select boarding house details for a specific house ID, including room information.

```
SELECT
  boarding_house_id,
  house_name,
  room_id,
  room_type,
  price,
  capacity,
  available_slots,
  city
FROM system_core.boarding_houses AS bh
JOIN system_core.rooms AS r
ON r.boarding_house_id = bh.boarding_house_id
JOIN system_core.locations AS l
ON l.location_id = bh.location_id
WHERE bh.status = 'approved'
AND r.status = 'available'
AND r.available_slots > 0;
```

Data Output:

boarding_house_id	house_name	room_id	room_type	price	capacity	available_slots	city
1	Bayang Comfort Boarding H...	2	Double	3500.00	2	1	Bayang
2	Kauwagan Student Lodge	2	Shared	2500.00	4	2	Kauwagan
3	Pobolacion Study House	3	Double	3000.00	2	2	Pobolacion 2
4	Matani Budget Rooms	4	Single	4000.00	1	1	Matani

Query 3: A query to select boarding house details for a specific house ID, including room information, and a summary row.

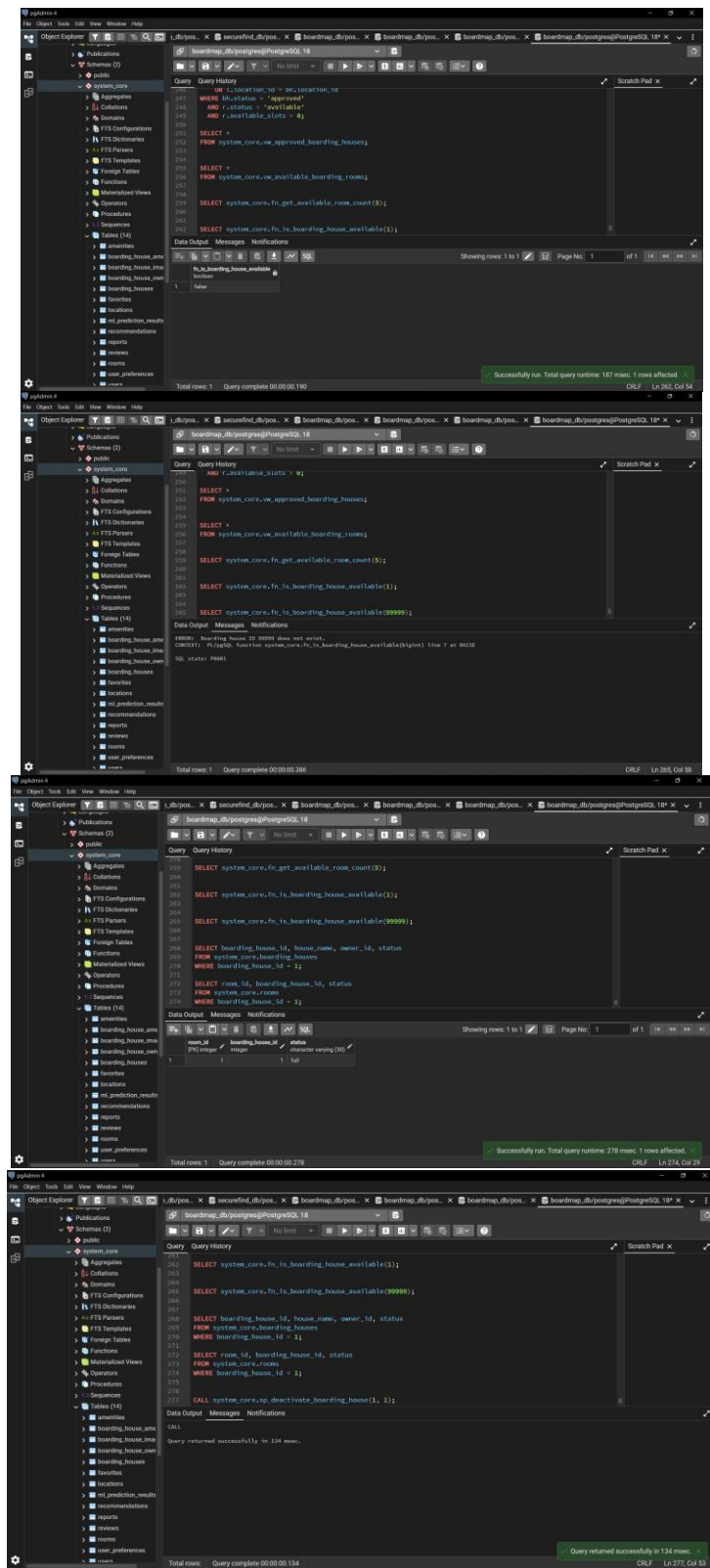
```
SELECT
  boarding_house_id,
  house_name,
  room_id,
  room_type,
  price,
  capacity,
  available_slots,
  city
FROM system_core.boarding_houses AS bh
JOIN system_core.rooms AS r
ON r.boarding_house_id = bh.boarding_house_id
JOIN system_core.locations AS l
ON l.location_id = bh.location_id
WHERE bh.status = 'approved'
AND r.status = 'available'
AND r.available_slots > 0;
```

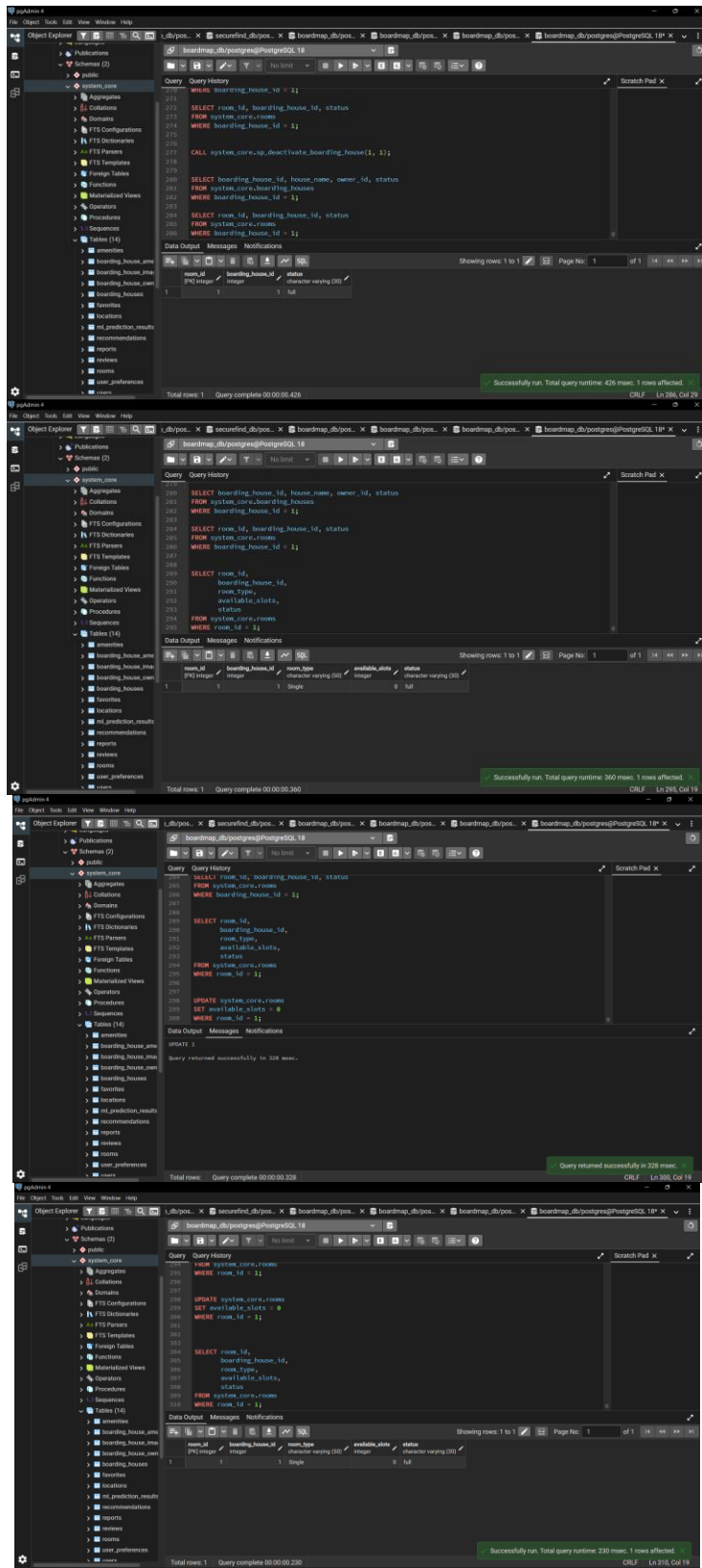
Data Output:

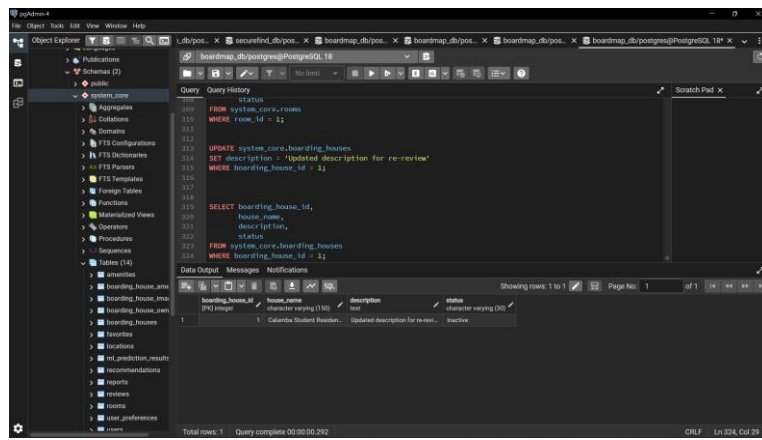
boarding_house_id	house_name	room_id	room_type	price	capacity	available_slots	city
1	Bayang Comfort Boarding H...	2	Double	3500.00	2	1	Bayang
2	Kauwagan Student Lodge	2	Shared	2500.00	4	2	Kauwagan
3	Pobolacion Study House	3	Double	3000.00	2	2	Pobolacion 2
4	Matani Budget Rooms	4	Single	4000.00	1	1	Matani

Summary Row:

board_id	board_name	board_status	board_available_rooms_count
1	Bayang Comfort Boarding H...	approved	2







Save the SQL scripts, test evidence, and documentation in the project repository, then submit the required files through MasaoLMS.

15. Testing and Evidence Worksheet

Use actual project objects. Every test must state the setup/input, expected behavior, actual outcome, and whether the evidence passes. Include screenshots or exported query results in the submitted evidence file where required by the instructor.

Test	Object / Rule	SQL / Input	Expected Result	Actual Result	Pass?
1	vw_approved_boarding_houses	SELECT * FROM system_core.vw_approved_boarding_houses;	Only approved houses; no sensitive credentials.	Not live-executed here; supplied sample data is expected to return approved listings only. Insert actual pgAdmin/DBeaver result.	Awaiting live execution
2	vw_available_boarding_rooms	SELECT * FROM system_core.vw_available_boarding_rooms;	Only approved houses with available rooms.	Not live-executed here; supplied sample data is expected to return available rooms in approved houses. Insert actual result.	Awaiting live execution
3	fn_get_available_room_count	SELECT system_core.fn_get_available_room_count(1);	Returns available slots for house 1.	Not live-executed here; supplied sample data indicates house 1 has 1 available slot. Verify live.	Awaiting live execution
4	fn_is_boarding_house_available	SELECT system_core.fn_is_boarding_house_available(1);	TRUE for approved house with available slots.	Not live-executed here; supplied sample data indicates TRUE for house 1. Verify live.	Awaiting live execution
5	fn_is_boarding_house_available missing	SELECT system_core.fn_is_boarding_house_available(99999);	Controlled not-found exception.	Not live-executed here; expected P0002 not-found exception. Insert actual error screenshot.	Awaiting live execution
6	sp_deactivate_boarding_house success	CALL system_core.sp_deactivate_boarding_house(1,1);	House and rooms become inactive.	Not live-executed here; expected house 1 and its rooms to become inactive. Use a development copy and	Awaiting live execution

				insert before/after evidence.	
7	sp_deactivate_boarding_house wrong owner	CALL system_core.sp_deactivate_boarding_house(1,999);	Ownership exception; no updates.	Not live-executed here; expected P0001 ownership exception and no data change. Insert actual error/before-after evidence.	Awaiting live execution
8	room status trigger	UPDATE system_core.rooms SET available_slots=0 WHERE room_id=1;	Room status becomes full.	Not live-executed here; setting available_slots=0 should result in status=full. Insert actual trigger evidence.	Awaiting live execution
9	listing reapproval trigger	UPDATE system_core.boarding_houses SET description='Updated' WHERE boarding_house_id=1 AND status='approved';	Listing status becomes pending.	Not live-executed here; changing a core field of an approved listing should result in status=pending. Insert actual trigger evidence.	Awaiting live execution
10	10	review integrity (UNIQUE + CHECK)	A) INSERT INTO system_core.reviews(user_id,boarding_house_id, rating,comment) VALUES (1,1,4,'Duplicate review test'); B) INSERT INTO system_core.reviews(user_id,boarding_house_id, rating,comment) VALUES (2,2,6,'Invalid rating test');	A) UNIQUE (user_id,boarding_house_id) rejects duplicate review. B) CHECK rating BETWEEN 1 AND 5 rejects rating 6.	Awaiting live execution

Minimum Test Coverage

•At least one positive and one failure/boundary case for each function or procedure where failure is meaningful. •At least one test proving each trigger fires at the correct event/timing and one check for unintended side effects. •At least one test verifying each view returns only the intended columns/rows and reflects project relationships correctly. •If a materialized view is implemented, demonstrate the difference between stale data and refreshed data. •Confirm that PostgreSQL constraints still protect integrity even when routines or triggers are bypassed by a direct SQL write where applicable.

16. Final Quality Checklist

Checklist	Checklist
[x] All submitted views, functions, procedures, and triggers use project specific names and actual semester-system rules.	[x] Every database object has a documented feature, consumer, or integrity purpose.
[x] At least two regular views are implemented and correctly tested.	[x] A materialized-view candidate is evaluated and its refresh/staleness decision is documented.

[x] At least two user-defined functions are implemented with appropriate parameters and return behavior.	[x] At least one stored procedure performs a meaningful project operation.
[x] At least two triggers are implemented with clear event/timing justification.	[x] Trigger logic does not duplicate declarative constraints or ordinary backend workflow without justification.
[x] At least one routine demonstrates meaningful exception/error handling.	[x] Database error messages are useful but do not expose sensitive information.
[x] The database-vs-API logic matrix analyzes at least five important rules.	[x] The authoritative layer for each critical rule is clear and defensible.
[x] At least ten reproducible tests cover normal, boundary, and failure behavior.	[x] SQL scripts can be executed in a clean project development database in dependency-safe order.
[x] Object-to-feature traceability agrees with the final project schema and current system requirements.	[x] Every group member can explain at least one object and one database-vs API boundary decision without reading a script.

17. Submission Requirements

- One SQL script or organized SQL folder containing the final project views, materialized view if used, functions, stored procedure(s), trigger functions, and triggers.
- Database-vs-API logic justification matrix with at least five project rules and a clear authoritative layer for each rule. •Object-to-feature traceability sheet connecting every submitted database object to an actual feature, report, requirement, or integrity rule.
- Testing evidence containing at least ten test cases with SQL/input, expected result, actual result, and pass/fail evidence.
- Short README or database-logic documentation explaining object creation order, dependencies, how to run tests, and any materialized-view refresh command.
- Files committed to the project GitHub repository under a folder such as database/logic/ or db/programmability/. •Repository link and required files submitted through MasaoLMS using the naming convention: IT108_LA5_Group##_SystemName. •Deliver a 5-7 minute group presentation using selected SQL and test results as evidence, followed by a 2-3 minute question-and answer period. Every member must participate meaningfully.

Academic Integrity and Competency Validation

The submitted database logic must represent the team's own semester system and implementation decisions. AI tools may be used to clarify syntax or concepts, but every SQL object must be reviewed, tested, understood, and defended by the group. The instructor may ask any member to explain a function, predict trigger behavior, identify an exception path, or justify why a rule belongs in PostgreSQL or the REST API/backend without assistance.

18. Scoring Rubric - 80 Points

Partial credit may be awarded when an object exists but is incorrect, unused, weakly justified, duplicated across layers, or unsupported by testing evidence.

Criterion	Evidence for Full Credit	Points
Views and materialized-view analysis	Two useful project views are correct and tied to real features; materialized-view use and refresh/staleness are justified.	10
PL/pgSQL functions	Functions use appropriate parameters, return types, PL/pgSQL structure, reusable project logic, and correct tested results.	14
Stored procedure	The procedure performs a meaningful multi-step database operation, handles affected data safely, and is correctly tested.	10
Triggers and trigger functions	Triggers use correct events/timing, implement justified integrity/audit/lifecycle behavior, and pass positive and side-effect tests.	14

Exception handling and integrity	Expected failures are handled meaningfully; constraints remain authoritative; errors are safe and useful.	8
Database-vs-API boundary justification	At least five rules have clear authoritative/supporting layers, sound reasoning, and no unexplained duplication.	10
Testing, traceability, and documentation	Ten or more reproducible tests, object-to-feature traceability, and execution notes are complete and consistent.	6
Group presentation and oral defense	Presentation is clear and within time; every member explains submitted SQL, trigger behavior, tests, and boundary decisions accurately.	8
Raw Score	____ / 80	Equivalent
Checked by	_____	Date

Remarks: All required database objects and boundary decisions are project-specific to BoardMap. Live SQL execution evidence remains the team's responsibility and must be inserted at the marked screenshot locations.

19. Post-Laboratory Reflection

Which project rule was the strongest candidate for PostgreSQL-side logic, and why?

Room availability/status synchronization is a strong PostgreSQL candidate because it is directly tied to row changes and should remain true regardless of whether data is changed through Flutter, the REST API, an admin tool, or another approved client. The trigger provides a final data-side safeguard while the API still performs user-friendly validation.

Which rule did your group deliberately keep in the REST API/backend layer, and what application context made that choice better?

The group deliberately keeps owner-only listing authorization in the REST API/backend. The backend has the authenticated user, role, and request/session context needed to determine whether the current owner is allowed to modify or deactivate a specific boarding house. PostgreSQL still protects ownership relationships and performs the data-side operation, but authentication and application authorization remain in the backend.

The listing re-approval trigger has the greatest hidden-side-effect risk because an UPDATE to an approved boarding house can automatically change its status to pending. The trigger should only watch core listing fields that materially affect verification, and the team should test that unrelated edits do not cause unnecessary re-review.

What failure case was hardest to test or handle in PL/pgSQL?

The hardest failure case is an owner attempting to deactivate a boarding house that exists but is owned by someone else. The procedure must reject the operation without changing the boarding house or its rooms. The error should be clear enough for the REST API to return an appropriate response.

If your project implemented a materialized view, how will you prevent users from relying on stale data? If you did not implement one, why not?

BoardMap does not implement a materialized view for the core map/search workflow because listing status and room availability can change and should be read from current data. The rating-summary materialized view is only a candidate for slower administrative reporting. If implemented later, the team must define a refresh command/schedule and document the acceptable staleness period.

vw_approved_boarding_houses is the database object most likely to be reused. It provides a stable approved-listing projection containing house, owner, and map-location information that can support Flutter map/search, REST API listing responses, administrative review, and future clients.

Before integration, the team should revisit the room-status trigger, listing re-approval trigger, procedure error codes, and the fields exposed by the views. The REST API should correctly interpret database outcomes—for example, an edited approved listing may become pending after the trigger fires—while ML execution and authentication/authorization remain in the backend.

Appendix A - Database Logic Inventory Summary

Inventory Item	Final Value
Regular views implemented	2
Materialized views implemented	0 implemented / 1 candidate evaluated
User-defined functions implemented	2
Stored procedures implemented	1
Triggers implemented	2
Test cases completed	10 planned; live execution pending
Database-vs-API rules analyzed	6
SQL script / folder name	database/logic/

vw_approved_boarding_houses is the most reusable because the same approved listing projection can support Flutter map/search, listing results, API endpoints, admin review, and future clients.

Review the room-status and listing-reapproval triggers, procedure error codes, and view exposure. The API must reflect database outcomes—for example, a listing edit can return pending after the reapproval trigger fires.

Final reminder: Put logic in PostgreSQL when it protects the data, improves reuse, or strengthens transaction integrity; keep application-specific orchestration and external-service behavior in the backend/API. The best design makes the boundary explicit and testable.F-CID-010 Rev.3, 03-